

Topic 2:

Data mapping and exchange

Agenda

- Introduction
- Why data mapping and exchange?
- Data representation and encoding
- XML technologies
 1. XML
 2. XML schemas (DTD and XML schema)
 3. Parsing XML documents
 4. Xpath
 5. XSL and XSLT
 6. Java XML APIs
- Examples
 1. How to create [Java XML application](#) (XML, DTD and Java APIs)
 2. How to create [Java XML Web application](#) (XML, XSL and XSLT)

NOTE: [these are two different types of application](#)
- Important resources
- Summary

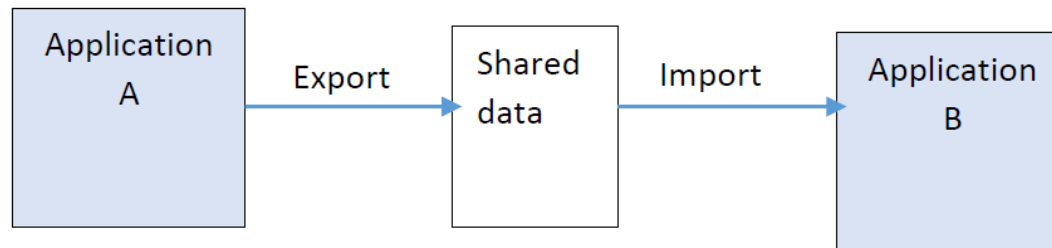
Introduction

- ❑ Most applications create, store, process and transfer data
- ❑ Also, applications work in heterogeneous environment:
 - i. Different applications (e.g., in different programming languages)
 - ii. Data in different formats
 - iii. Different hardware and software
 - iv. Etc.
- ❑ Integrative programming and technologies is important:
 - Allows **different** applications to work together
 - One way is to allow applications to **share data**

Introduction

❑ Common data sharing techniques

1. **File transfer:** one application writes data to a file; another reads from the file. The file can be binary, text, XML, Excel, etc.



Data transfer

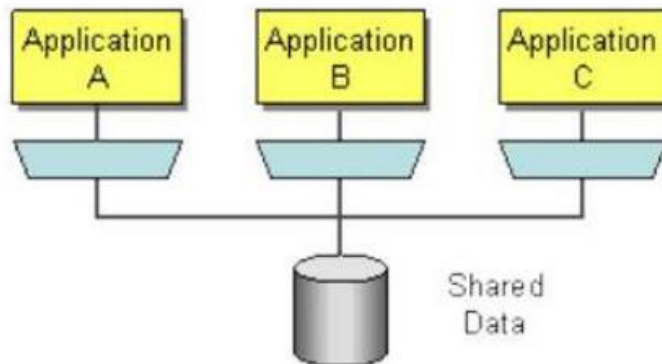
Question: Can there be any problem with data exchange among the applications?

✓ **Question:** any problem which can arise?

Introduction

❑ Common data sharing techniques

2. **Shared database:** applications share data in a database

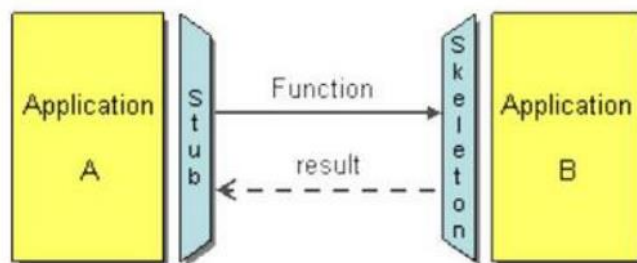


✓ Any problem(s) which can arise?

Introduction

- ❑ Common data sharing techniques
 - ❑ Applications on a network can share data
 - ❑ On a network?

3. **Remote Procedure Call** (or Remote Method Invocation): A remote application calls the procedures (i.e., functions or methods) “exposed” by another application.



Introduction

- ❑ Why is it important for different applications to share data?
- ❑ Why is a **standard** for creating, storing, processing, and transferring data important?

Introduction

- ❑ Common problem in software development: **storing, accessing, manipulating** and **displaying** data
 - Data can be stored in different formats, e.g., in
 1. Files (text, binary, Excel, XML, etc.)
 2. Databases (relational, object, etc.)
 - Data can be accessed and manipulated in different ways, e.g.,
 - Different programming languages
 - Heterogeneous network
 - Data can be displayed in different applications
 - E.g., browsers: Chrome, Firefox, Bing, etc.
- ❑ Therefore, an important part of software development is to format and exchange data across different platforms

Introduction

- ❑ There are several standards
- ❑ Standards ensure
 1. Rules and guidelines, e.g., data is formatted and transferred in a particular way (i.e., specific encoding and decoding)
 2. Vendor, platform, etc., independence
 3. Interoperability (exchange between different applications)
 4. Efficiency? Why
- ❑ Examples of standards
 - ❑ SGML (Standard Generalized Markup Language)
 - ❑ HTML
 - ❑ XML
 - ❑ JSON
 - ❑ Etc.
- ❑ Exercise: Compare XML and JSON (i.e. advantages and disadvantages)

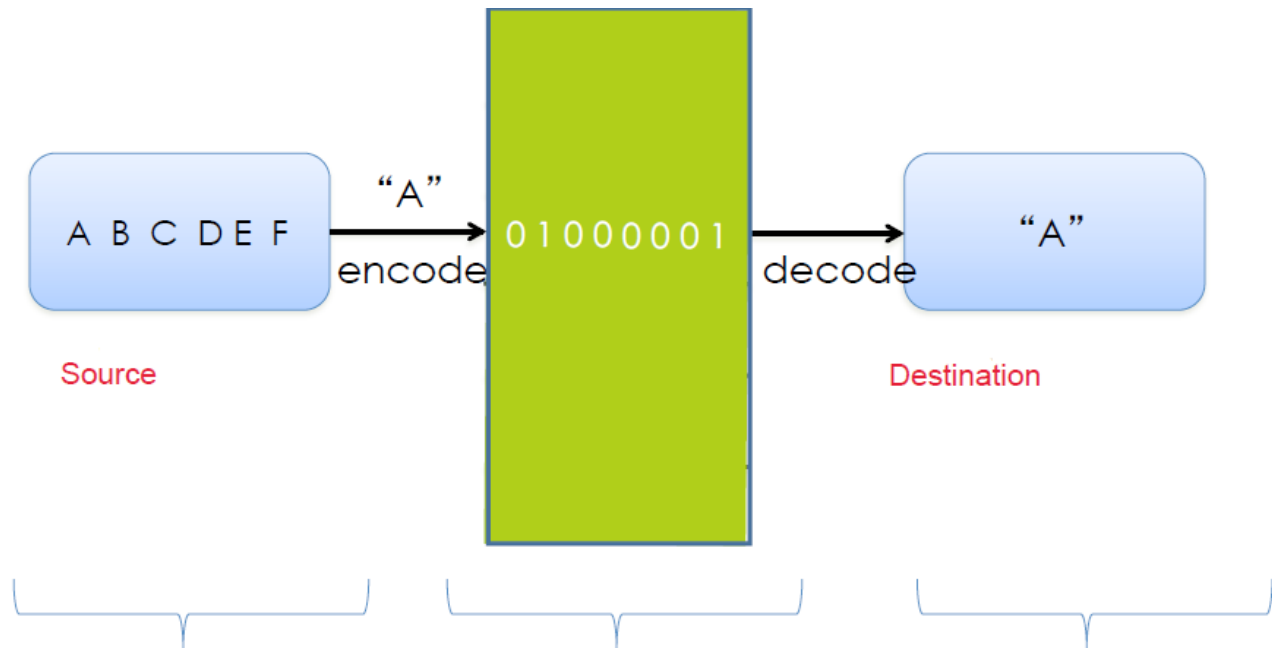
Why mapping and exchange?

❑ An understanding can enable

1. Openness:
 1. independent of hardware and software
 2. Free
2. Increase interoperability
3. Increase reuse
4. Increase maintainability
5. Reduced total development cost
6. Etc.

Data representation and encoding

- ❑ There are several encoding standards
 - ❑ ASCII
 - ❑ Unicode 8 (UTF-8), Unicode 16 (UTF-16), Unicode 32 (UTF-32)
 - ✓ UTF: Unicode Transformation **Format**
- ❑ Example: encoding and decoding in UTF-8



External representation Internal representation External representation

Data representation and encoding

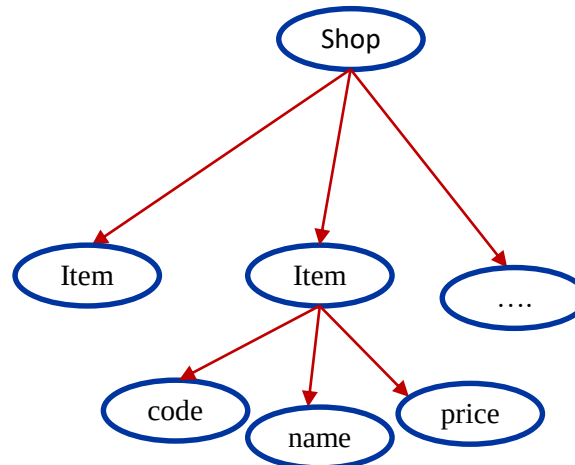
❑ Common data structures

1. Relational: uses table, rows and columns to structure data.

2. E.g., [Excel](#) and [Databases](#)

Code	Name	Price
001	Bread	12.99
002	Rama	40.99
003	Coke	16.49

3. Tree: uses tree to structure data; hierarchical; non-linear



XML Technologies

- ❑ XML: Extensible Markup Language is a standard for:
 1. Data storage (e.g., structured data in a file)
 2. Data processing (e.g., same format for different applications)
 3. Data exchange between systems (e.g., using the Web)
- ❑ XML is a group of technologies which includes:
 1. XML (the markup language)
 2. XML schemas: to define the structure of an XML document: e.g., DTD (Data Type Definition) and XML schema
 3. XSL (Extensible Stylesheet Language) and XSLT (XSL Transformation): for presentation (to display data)
 4. XML Parser: reads and validates XML documents; applications (e.g., Web browsers) contain parsers
 5. XPath: used to locate elements and attributes in XML documents
 6. XQuery: used to extract elements and attributes from XML documents; built on XPath
 7. APIs: libraries (Java, C#, Python, etc.) to access and manipulate XML.

XML

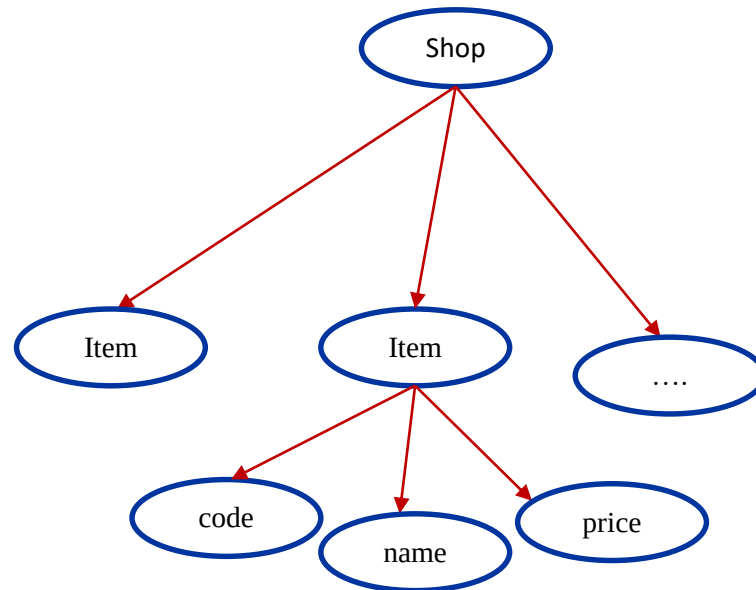
- ❑ XML uses **tags** to structure data (like HTML)
- ❑ XML document with correct syntax is called **valid** (or **well-formed**)
- ❑ XML syntax rules
 1. Prolog (declaration)
 - Optional but if used, it must be the 1st line
 - UTF 8 is the default in English speaking countries
 2. Only one root element
 3. Tags are case-sensitive (**name** and **Name** are different)
 4. Each element must have closing tag
 5. Elements must be properly nested
 6. Attributes must be quoted
- ❑ Prolog is not a tag (so **no closing tag** required)

XML

- ❑ Example: consider a shop with many items, each item has **code**, **name** and **price**
- ❑ Relational versus Tree structures

Code	Name	Price
001	Bread	12.99
002	Rama	40.99
003	Coke	16.49

vs



XML

- ❑ XML Example: consider a shop with many items, each item has code, name and price
- ❑ Relational versus XML
 - Note: XML code shows only 1 item
 - Note the difference between a tag and element

Code	Name	Price
001	Bread	12.99
002	Rama	40.99
003	Coke	16.49

1st line: XML declaration

```
<?xml version="1.0" encoding="UTF-8"?>
<!--Example:a Shop (root) has many Items -->
<Shop>
  <!--An Item has code, name and price -->
    <Item>
      <code>0001</code>
      <name>Bread</name>
      <price>12.99</price>
    </Item>
  </Shop>
```

Comment

Elements

XML

❑ Note: we can use attributes in a tag

○ Example:

```
<Item code="001">  
    <name>Bread</name>  
    <price>12.99</price>  
</Item>
```

❑ Rules for attributes

1. Only one attribute/value pair
2. Cannot contain tree structure

❑ When to use element or attribute: it depends but elements have advantages

- You break data into two or more parts as elements but not 2 or more attributes

XML validation

- ❑ How to check if XML document is well-formed (i.e., valid)
 1. In an IDE (e.g., Eclipse)
 2. Online tools

XML

❑ Exercise 1

1. Write the XML code to store the data in a file called **assesement.xml**. The root node is **Mark** and contains **Student** nodes. Each Student node contains **studid**, **dp** and **exam** with the values below.

studid	dp	exam
2020202050	50	
2020202140	60	
2020202275	75	

We'll build on this, so complete this before the next lecture

XML and Java application

XML schemas

- ❑ There are two main types of schemas
 1. DTD (Data Type Definition)
 - Simpler
 - File extension: [.dtd](#) (e.g., [item.dtd](#))
 2. XML Schema
 - Comprehensive (there are many features which are not in DTD)
 - File extension [\(.xsd](#) e.g., [item.xsd](#))
- ❑ There are various tools to generate schemas (DTD or XML Schema) from XML documents
 1. IDEs
 2. Online tools

How to use XML and DTD

❑ Example: XML for shop (many items)

1. Create the XML (and save as **items.xml**)

```
<?xml version="1.0" encoding="UTF-8"?>
<Shop>
  <Item>
    <code>001</code>
    <name>Bread</name>
    <price>12.99</price>
  </Item>
  <Item>
    <code>002</code>
    <name>Rama</name>
    <price>40.99</price>
  </Item>
  <Item>
    <code>003</code>
    <name>Coke</name>
    <price>16.49</price>
  </Item>
</Shop>
```

How to use XML and DTD

2. Create DTD for the shop and items example (the structure can be different)

This goes into the XML document

DTD Example 1

```
-->

<!-- An example how to use this DTD from your XML document:
      <?xml version="1.0"?>
      <!DOCTYPE Shop SYSTEM "items.dtd">
      <Shop>
      ...
      </Shop>
-->

<!ELEMENT Shop (Item)*>
<!ELEMENT Item (code,name,price)*>
<!ELEMENT code (#PCDATA)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT price (#PCDATA)>
```

Occurrence of elements: options (remember constraints in DBMS?)

None (exactly 1 occurrence)

? (0 or 1 occurrence)

* (>=0 occurrence)

+ (>=1 occurrence)

How to use XML and DTD

DTD Example 2: same as Example 1
but contain more details

Notes:

- 1. CDATA: Unparsed character data
(Not parsed; not expanded)
- 1. PCDATA: Parsed character data
(Text between start and end tags)
- 1. ATTLIST: Attribute list

Occurrence

```
<?xml encoding="UTF-8"?>
<!ELEMENT Shop (Item)+>
<!ATTLIST Shop
    xmlns CDATA #FIXED ''>

<!ELEMENT Item (code,name,price)>
<!ATTLIST Item
    xmlns CDATA #FIXED ''>

<!ELEMENT code (#PCDATA)>
<!ATTLIST code
    xmlns CDATA #FIXED ''>

<!ELEMENT name (#PCDATA)>
<!ATTLIST name
    xmlns CDATA #FIXED ''>

<!ELEMENT price (#PCDATA)>
<!ATTLIST price
    xmlns CDATA #FIXED ''>
```

How to use XML and DTD

3. Linking the DTD file to the XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE Shop SYSTEM "items.dtd">
<Shop>
  <Item>
    <code>001</code>
    <name>Bread</name>
    <price>12.99</price>
  </Item>
  <Item>
    <code>002</code>
    <name>Rama</name>
    <price>40.99</price>
  </Item>
  <Item>
    <code>003</code>
    <name>Coke</name>
    <price>16.49</price>
  </Item>
</Shop>
```

Links XML to the DTD file

How to use XML and DTD

❑ Exercise 2

1. Write the XML code to store the data in a file called **assesement.xml**. The root node is **Assessment** and contains **Student** nodes. Each Student node contains **studid**, **dp** and **exam** with the values below (See Exercise 1)

studid	dp	exam
2020202050	50	
2020202140	60	
2020202275	75	

2. Create a DTD for the XML document
3. Link the XML to the DTD

XML and XML Schema

❑ Exercise: Use the **Shop** example

1. Generate an XML schema (extension .xsd)
2. Compare XML schema to the DTD

How to create Java XML application

- ❑ You can write a Java program to work with an XML document linked to DTD
- ❑ Java APIs for XML
 - DOM (Document Object Module)
 1. Represents XML structure as tree
 2. Loads entire XML document into memory (memory intensive)
 3. Allows read and write
 4. Only for small documents (<10 megabytes)
 - SAX (Simple API for XML)
 1. Memory efficient; faster
 2. Allows read only
 3. Push (only load the data as and when necessary to the application)
 - StaX (Streaming API for XML)
 1. Improvement on DOM and SAX
 2. Memory efficient; faster
 3. Easier to use
 4. Any size of document (small and large)

How to create Java XML application

- ❑ StaX provides two main APIs
 1. The Cursor API
 - XMLStreamReader (to read)
 - XMLStreamWriter (to write)
 2. The Iterator API
 - XMLEventReader (to read)
 - XMLEventWriter (to write)
- ❑ We will use the Cursor API in our example

How to create Java XML application

❑ Example:

1. Write code to read the data in an XML document (e.g., items.xml) as objects and store in a list
2. Write code to test (i.e., display the objects in the list above)

❑ Note: because we'll deal with **Item** objects, we need a Item class

❑ Use **the Cursor API**

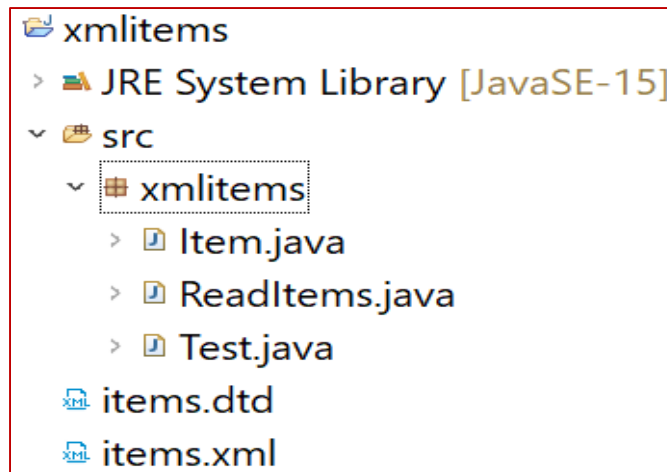
1. XMLStreamReader (to read)
 - We'll need to set up a **stream** to "**connect**" the application to the XML file
 - Read the objects
 - Close the **file** and **stream**
- ~~2. XMLStreamWriter (to write)~~ **We will not write to XML file in this example**

❑ Required: **try/catch for any exceptions** (in case something goes wrong, e.g., file does not exist)

How to create Java XML application

❑ Example:

1. Write POCO for objects of Item ([Item.java](#))
2. Write code to read the data in the XML document (e.g., items.xml) as objects and store in a list ([ReadItem.java](#))
3. Write code to test, i.e., display the objects in the list above(i.e., [Test.java](#))



Note: see the Creating XML Java applications

How to create Java XML application

- ❑ Reading the data of **Item** objects in the XML file
 1. Handle **Exception**
 2. Create a list to store all the **Item** objects (records) in the XML file
 3. Open the XML file
 4. Create XMLReader object to connect to the XML file
 5. Check:
 - a. if end of the file, close the stream and file
 - b. Otherwise, read the next “record”
 6. Reading a record: based on the “event” take an action
 - a. If “event” is the start element of Item : create a new Item
 - b. If “event” is “text” (i.e., value), store the text:
 - c. If “event” is close element of:
 - i. **code**, **name**, **price**: set the appropriate value
 - ii. **Item** add the record to the list

How to create Java XML application

❑ Exercise

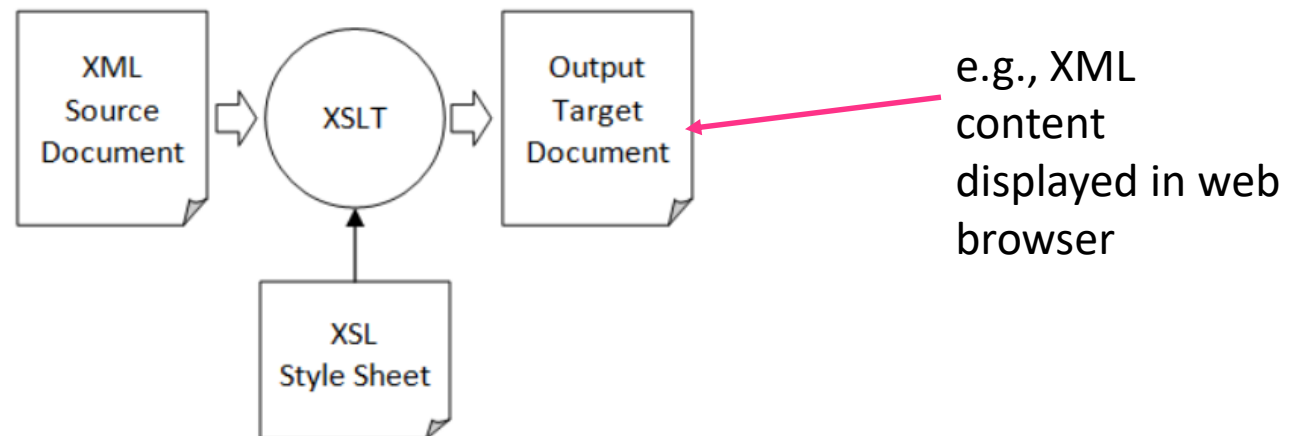
1. Create an XML document (**assessment.xml**) which contains **Assessment** (root) and at least three **Student** records. Each **Student** record consists of: **id** (a **unique identifier**); **name** (name of the student); **dp** (dp obtained in the module) and **exam** (exam mark obtained in the module).
2. Create a DTD (**assessment.dtd**) and link to **assessment.xml**
3. Write the following Java classes
 1. **Student**: for student objects in the XML
 2. **ReadAssessment** : to read all Student objects from XML file into a list
 3. **Tester**: print all the Student objects in the list

❑ NOTE: **complete Exercise 2: Java XML application**

How to create Java XML Web application

How to create Java XML Web application (XML, XSL and XSLT)

- ❑ XSLT transform XML documents into HTML, XHTML plain text, etc.
- ❑ How they work together
 1. Create XML document
 2. Create XSL document (how the XML should display)
 3. XSLT transforms the XML based on the XSL to display in the target application (e.g., browser)



How to create Java XML Web application (XML, XSL and XSLT)

❑ In this section, we'll create a web application

- This is based on the Shop example (shop with many items) and
- Displays data in an XML document in a web page

List of items

Code	Name	Price
001	Bread	12.99
002	Rama	40.99
003	Coke	16.49

❑ Steps:

1. Create XML document ([items.xml](#))
2. Generate the XSL document ([items.xsl](#)) from the XML file ([item.xml](#))
3. Link the XSL document ([items.xsl](#)) to the XML file (items.xml)
4. Run the XML file ([items.xml](#))
 1. On a server or
 2. Open XML document in a web page (most browsers, except Firefox, do not display the web page)
 3. Ctrl+O → Select XML file

Note: use the internal browser in Eclipse

- Window → Preference → General → Web browser → Internal browser)
- See

How to create Java XML Web application (XML, XSL and XSLT)

- ❑ Example: shop (with many items)
- ❑ Step1: Create XML document ([items.xml](#))

```
<?xml version="1.0" encoding="UTF-8"?>
<Shop>
  <Item>
    <code>001</code>
    <name>Bread</name>
    <price>12.99</price>
  </Item>
  <Item>
    <code>002</code>
    <name>Rama</name>
    <price>40.99</price>
  </Item>
  <Item>
    <code>003</code>
    <name>Coke</name>
    <price>16.49</price>
  </Item>
</Shop>
```

How to create Java XML Web application (XML, XSL and XSLT)

- ❑ Example: shop (with many items)
- Step 2: Generate the XSL document ([items.xsl](#)) for the XML file ([item.xml](#))

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="2.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/">
    <!-- TODO: Auto-generated template -->
    <h2>List of items</h2>
    <hr/>
    <table border="1">
      <tr>
        <th>Code</th>
        <th>Name</th>
        <th>Price</th>
      </tr>
      <xsl:for-each select="Shop/Item">
        <tr>
          <td><xsl:value-of select="code"/></td>
          <td><xsl:value-of select="name"/></td>
          <td><xsl:value-of select="price"/></td>
        </tr>
      </xsl:for-each>
    </table>
  </xsl:template>
</xsl:stylesheet>
```

List of items

Code	Name	Price
001	Bread	12.99
002	Rama	40.99
003	Coke	16.49

How to create Java XML Web application (XML, XSL and XSLT)

- ❑ Example: shop (with many items)
- ❑ Step 3: Link the XSL document ([item.xsl](#)) to the XML file ([items.xml](#))

```
<?xml-stylesheet type="text/xsl" href="items.xsl"?>
<Shop>
  <Item>
    <code>001</code>
    <name>Bread</name>
    <price>12.99</price>
  </Item>
  <Item>
    <code>002</code>
    <name>Rama</name>
    <price>40.99</price>
  </Item>
  <Item>
    <code>003</code>
    <name>Coke</name>
    <price>16.49</price>
  </Item>
</Shop>
```

How to create Java XML Web application (XML, XSL and XSLT)

❑ Example: shop (with many items)

❑ Step 4: Run the XML file ([item.xml](#))

1. To run on a server in Eclipse:
 - Right-click the project (or on the XML file)
 - Select the Run on server
 - Select the server to run on
2. To run in a browser

How to create Java XML Web application (XML, XSL and XSLT)

❑ Exercise

1. Create an XML document (**assessment.xml**) which contains **Assessment** (root) and at least three **Student** records. Each **Student** record consists of: **id** (a **unique identifier**); **name** (name of the student); **dp** (dp obtained in the module) and **exam** (exam mark obtained in the module).
2. Create an XSL document (**assessment.xsl**) and link to **assessment.xml** to display the data in a web page

NOTE: [see Exercise 3: Java XML Web application](#)

Important resources

- Streaming API:
<https://docs.oracle.com/javase/tutorial/jaxp/stax/why.html>
- Murach's Java Programming (4th Edition): Chapter 10

Summary

- Why data mapping and exchange?
- Data representation and encoding
- XML technologies
 1. XML
 2. XML schemas (DTD and XML schema)
 3. Parsing XML documents
 4. Xpath
 5. XSL and XSLT
 6. Java XML APIs
- Examples
 1. Java XML application (XML, DTD and Java APIs)
 2. XML web application (XML, XSL and XSLT)
- Important resources

Comments and feedback

?

Next.....

- Intersystem communication